

Seralix: A Minimal Language Prototype for Expressing Fast Fourier Transform Algorithms

Logan Taurus
Universidad de los Andes
Systems and Computing Engineering

Abstract—This paper presents Seralix, a minimal programming language prototype designed to explore language construction through the implementation of the Fast Fourier Transform (FFT). The project demonstrates how a custom language can be developed to express non-trivial algorithms while maintaining simplicity in syntax and execution. Rather than aiming to replace existing languages, Seralix serves as a proof of capability in language design, parsing, and interpretation. The implementation of recursive FFT and polynomial multiplication validates both the correctness of the language runtime and its ability to express complex computational procedures.

I. INTRODUCTION

The Fast Fourier Transform (FFT) is a fundamental algorithm in computer science, widely used in signal processing, numerical analysis, and polynomial multiplication. Its implementation requires careful handling of recursion, numerical precision, and structured computation.

While modern programming languages provide built-in abstractions for such algorithms, the internal mechanisms of language design are often hidden from developers. This project explores the construction of a programming language by implementing Seralix, a minimal prototype capable of expressing and executing FFT-based algorithms.

The primary objective is not to create a production-ready language, but to demonstrate that the author is capable of designing and implementing a complete language pipeline, including lexical analysis, parsing, abstract syntax tree construction, and execution.

II. PROBLEM STATEMENT

Implementing FFT requires:

- Recursive decomposition
- Complex arithmetic
- Efficient data manipulation
- Structured control flow

Traditional languages provide these features implicitly. This project investigates whether a minimal custom language can support such requirements while maintaining a simple and consistent syntax.

The central question addressed is:

Can a minimal language prototype be designed and implemented to correctly express and execute FFT-based algorithms?

III. SYSTEM ARCHITECTURE

Seralix follows a classical interpreter architecture:

Source Code $\rightarrow$ Lexer $\rightarrow$ Parser $\rightarrow$ AST $\rightarrow$ Interpreter

A. *Lexer*

The lexer transforms raw source code into tokens, including identifiers, literals, keywords, and symbols.

B. *Parser*

The parser constructs an Abstract Syntax Tree (AST) using a recursive descent-like approach with operator precedence handling.

C. *Interpreter*

The interpreter evaluates the AST using a dynamic runtime environment, supporting:

- function calls
- control flow
- arithmetic operations
- built-in functions

IV. LANGUAGE DESIGN

Seralix combines functional and imperative paradigms.

A. *Functions*

Two types of functions are supported:

Arrow functions:

$(x) \rightarrow x * x$

Named functions:

```
function add(x, y=2) {  
    return x + y;  
}
```

B. *Control Flow*

```
if condition {  
    ...  
} elif other {  
    ...  
} else {  
    ...  
}
```

C. Loops

```
while condition {  
    ...  
}  
  
for x in range(5) {  
    ...  
}
```

D. Expressions

Seralix supports arithmetic, bitwise, boolean, and ternary expressions.

```
x * y % 2 == 0 ? 'even' : 'odd'
```

V. FFT IMPLEMENTATION IN SERALIX

To validate the language, a recursive FFT algorithm was implemented.

A. Recursive Structure

```
function fft(a, invert=false) {  
    if len(a) == 1 {  
        return [a[0]];  
    }  
  
    even = fft(a[0::2], invert);  
    odd = fft(a[1::2], invert);  
    ...  
}
```

This demonstrates that the language supports recursion and sequence manipulation.

B. Polynomial Multiplication

Polynomial multiplication is performed using FFT:

```
function polymul(a, b) {  
    ...  
}
```

The algorithm consists of:

- 1) Padding inputs to a power of two
- 2) Applying FFT to both sequences
- 3) Pointwise multiplication
- 4) Inverse FFT

VI. RESULTS AND VALIDATION

The correctness of the implementation was validated using known polynomial products.

Example:

```
a = [1, 2, 3];  
b = [4, 5, 6];  
  
print(polymul(a, b));
```

Output:

```
[4, 13, 28, 27, 18]
```

This matches the expected result:

$$(1 + 2x + 3x^2)(4 + 5x + 6x^2) \quad (1)$$

VII. DISCUSSION

The results confirm that:

- The language can express recursive algorithms
- The interpreter executes non-trivial computations correctly
- The syntax is sufficient for mathematical operations

However, several limitations were identified:

- Lack of efficient list operations (e.g., append)
- Floating-point precision errors in FFT
- Absence of integer division operator

These limitations highlight areas for future improvement.

VIII. CONCLUSION

This project demonstrates that a minimal programming language can be designed and implemented to execute complex algorithms such as FFT.

Seralix successfully provides:

- A functional parsing and execution pipeline
- Support for recursion and structured control flow
- A working implementation of polynomial multiplication using FFT

More importantly, the project validates the author's ability to construct a language system from first principles.

IX. FUTURE WORK

Future extensions of Seralix may include:

- Static type system
- Module and import system
- Optimized runtime
- Compiler backend
- Integration with AI frameworks (Seraphine)